

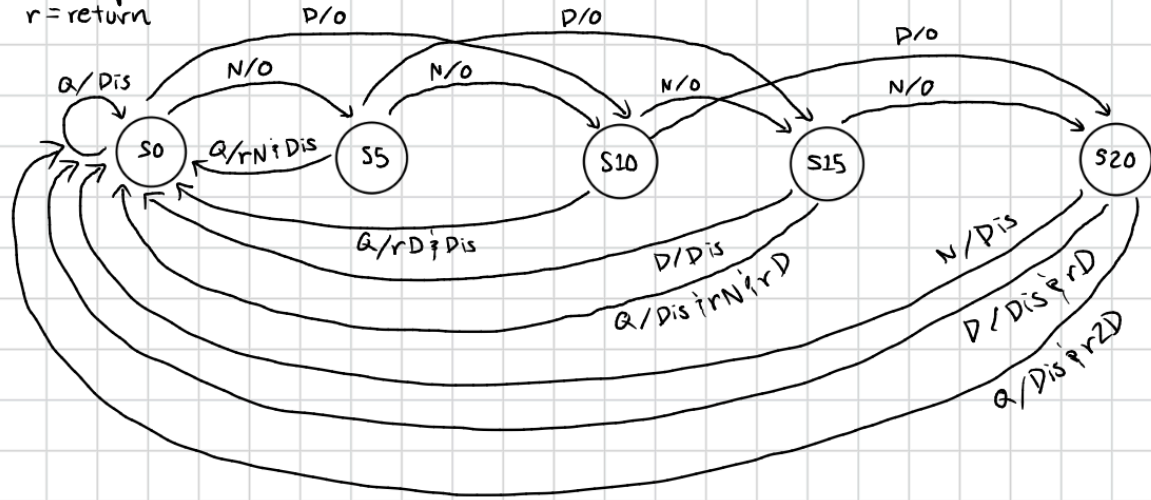
DESIGN & CALCULATIONS

Truth Table:

Present State	Bin Code	Input	Next State	Bin Code	Dispense	Return Nickel	Return Dime	Return Two Dimes
S0	000	N	S5	001	0	0	0	0
S0	000	D	S10	010	0	0	0	0
S0	000	Q	S0	000	1	0	0	0
S5	001	N	S10	010	0	0	0	0
S5	001	D	S15	011	0	0	0	0
S5	001	Q	S0	000	1	1	0	0
S10	010	N	S15	011	0	0	0	0
S10	010	D	S20	100	0	0	0	0
S10	010	Q	S0	000	1	0	1	0
S15	011	N	S20	100	0	0	0	0
S15	011	D	S0	000	1	0	0	0
S15	011	Q	S0	000	1	1	1	0
S20	100	N	S0	000	1	0	0	0
S20	100	D	S0	000	1	1	0	0
S20	100	Q	S0	000	1	0	0	1

Mealy Transition Diagram:

* Mealy FSM: input/output vectors, 3 outgoing for each state
 Dis = dispense
 r = return



Logic Equations: (with 101, 110, 111 as invalid/ don't care states)

Dis=Dispense, N=Nickel, D=Dime, Q=Quarter, r=return

State Bits: $S[2:0] \rightarrow S[2]=A, S[1]=B, S[0]=C$

States (Symbolic)	Binary (S[2:0])	Total Cents
S0	000	0
S5	001	5
S10	010	10
S15	011	15
S20	100	20

Output Equations: (Dis, rN, rD, r2D) $\rightarrow$ depend on current state & input

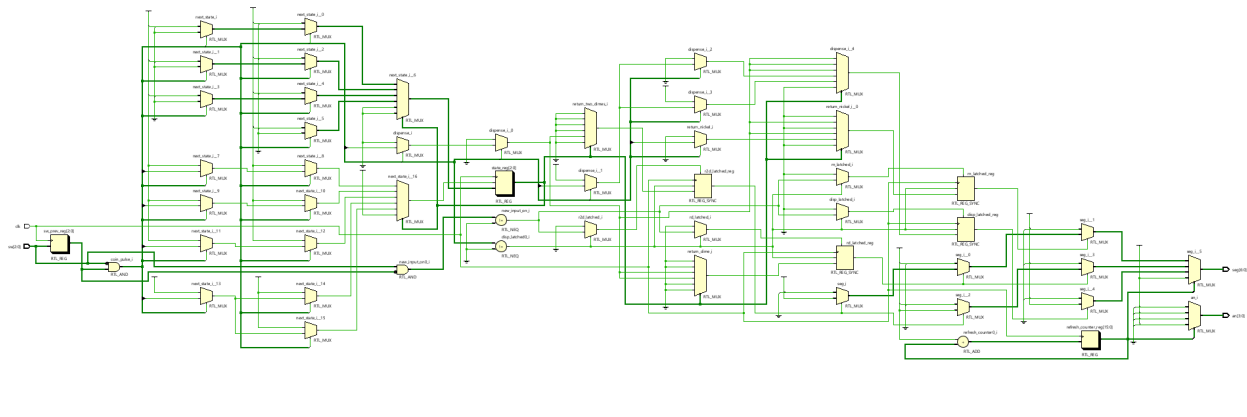
- Dis = $Q + D(S15 + S20) + N*(S20) = Q + D(A'BC + AB'C') + N(AB'C')$
 - $= Q + D(A + BC) + NA$
- rN = $Q*(S5 + S15) + D*(S20) = Q(A'B'C + A'BC) + D(AB'C')$
 - $= QC + DA$
- rD = $Q*(S10 + S15) = Q(A'BC' + A'BC)$

- = QB
- r2D = Q*(S20) = Q*(AB'C')
- = QA

Next-State Equations: (A+, B+, C+) → also depends on input & current state

- $A^+ = D(S10) + N(S15) = D*(A'BC') + N*(A'BC)$
 - = $A'B*(DC' + NC)$
- $B^+ = D(S0) + N(S5) + D(S5) + N(S10) = D(A'B'C') + N(A'B'C) + D(A'B'C) + N(A'BC')$
 - = $A'(D*B'C' + N*B'C + D*B'C + N*BC')$
- $C^+ = N(S0) + D(S5) + N(S10) = N(A'B'C') + D(A'B'C) + N(A'BC')$
 - = $A'(N*C' + D*B'C)$

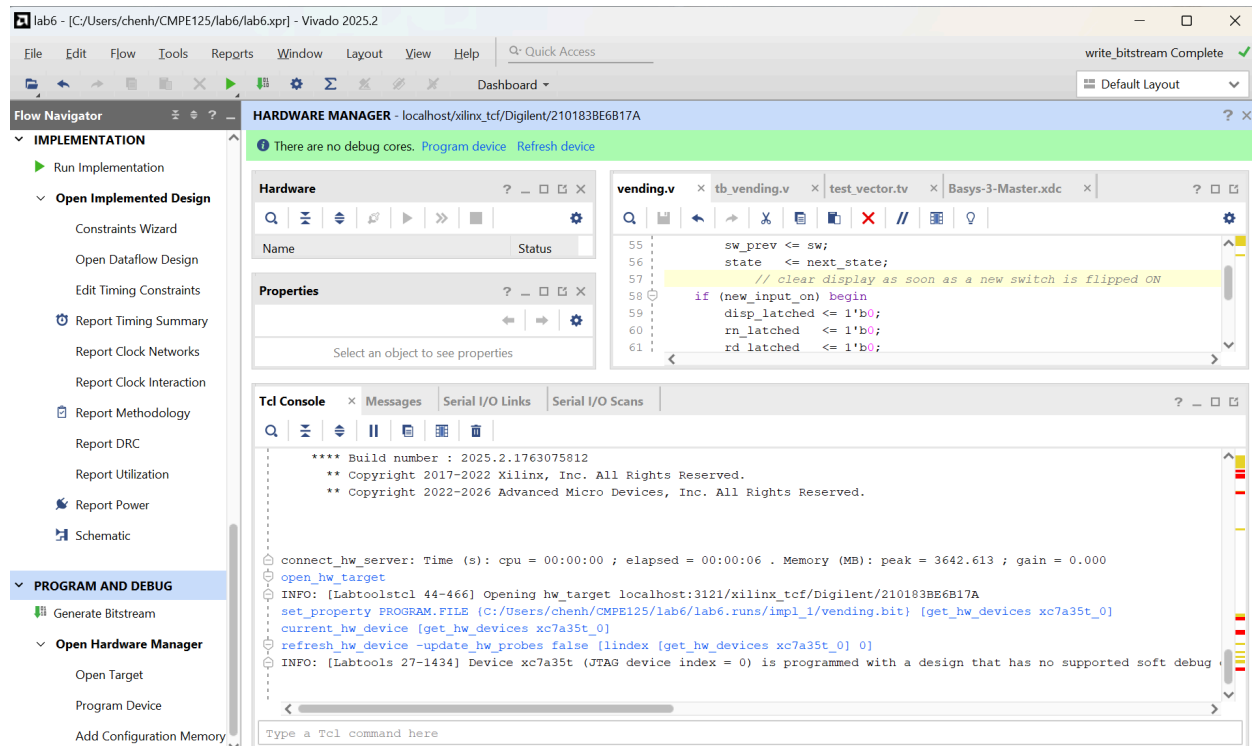
Circuit/ Block Diagrams:



Vivado-Generated Schematic

SCREENSHOTS

Compilation:



Bitstream generated successfully

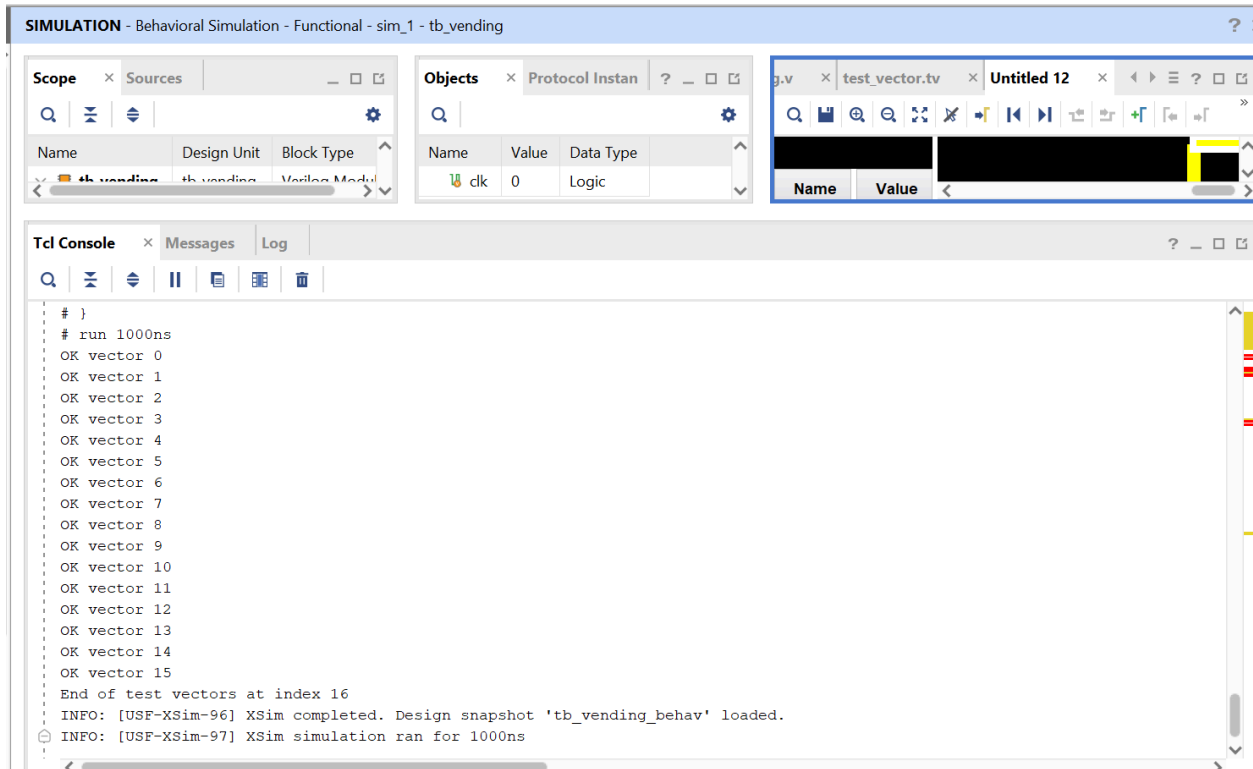
Simulation Waveforms:

- Vector format:

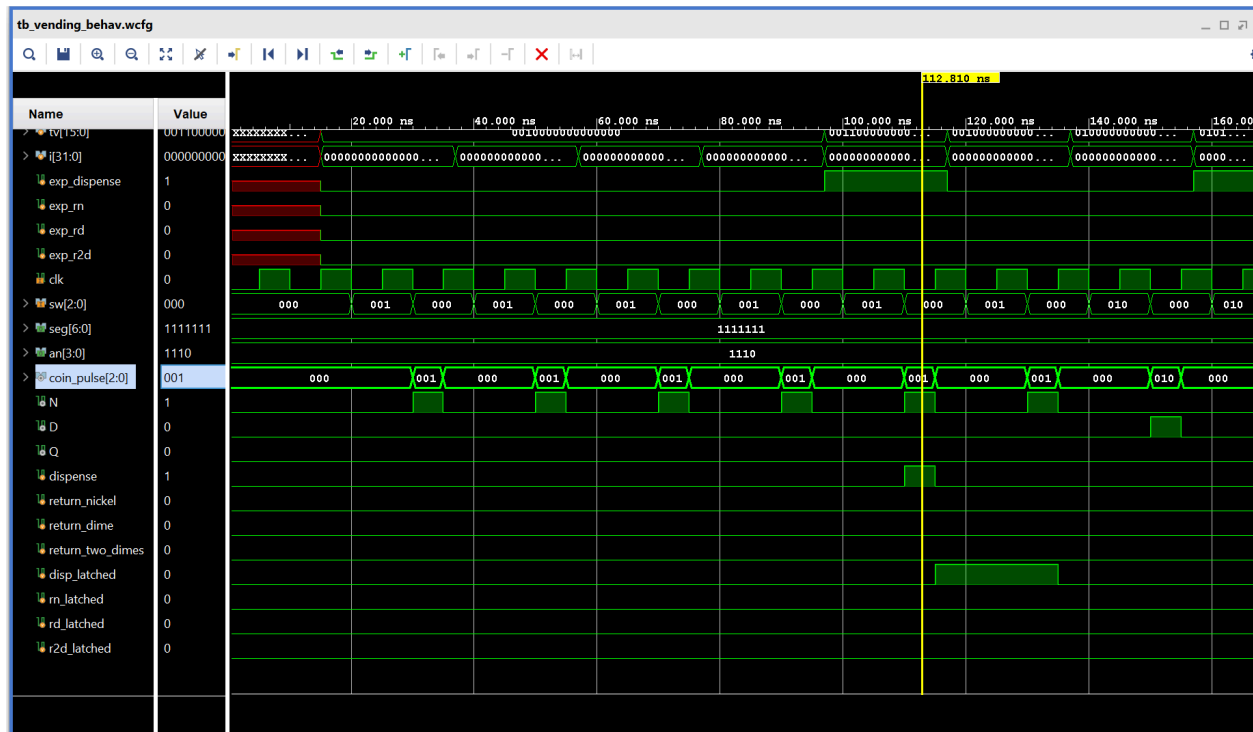
Bits	[15:13]	[12]	[11]	[10]	[9]	[8:0]
	Coin	D	rN	rD	r2D	Don't care

- Coins use separate signals, not encoded bits: n=001, d=010, q=100

Console Log Messages:

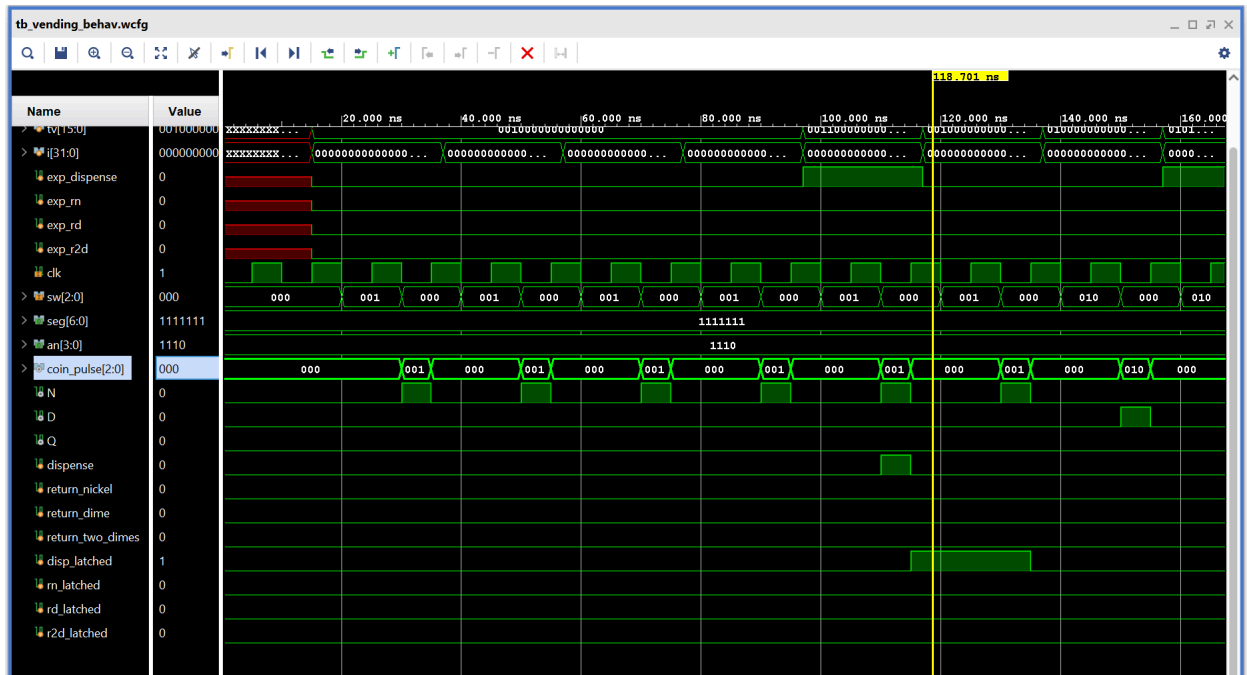


Waveform:



- 5 nickels inserted at ~112 ns (Coin pulse = 001 → nickel inserted)
- Coin pulse = 000 → represents coin dropping into machine/ being registered

- $Dispense = exp_dispense = 1$ (on the 25th cent)



- $Disp_latched=1$ to keep dispatch display ON on Basys3 board/ capture output
- When $coin_pulse=000 \rightarrow$ no coin being inserted
- Switch goes up $\rightarrow sw_prev$ updated at posedge $clk \rightarrow$ switch down $\rightarrow coin_pulse$ set to switch & coin is registered $\rightarrow$ vending machine computes/ updates outputs + latches

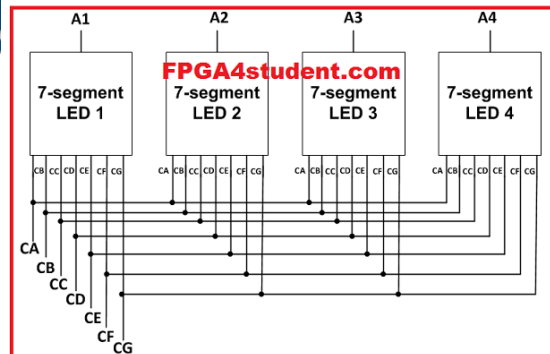
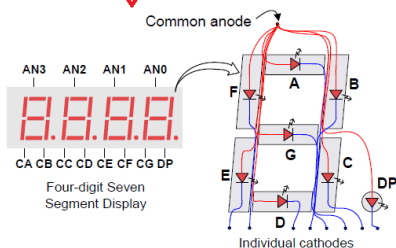
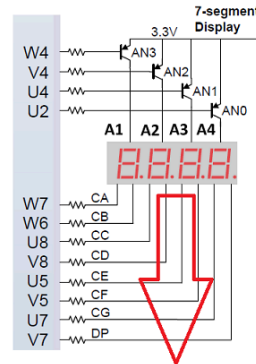
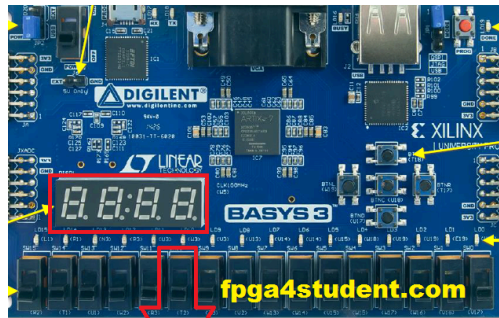
SUMMARY

Paragraph describing verification of implemented logic with the simulation results:

The goal of this lab is to design a vending machine with inputs (nickels, dimes, quarters) and outputs (Dispense, ReturnNickel, ReturnDime, ReturnTwoDimes). Upon reaching 25 cents in total value inserted, the vending machine will choose to dispense and return depending on the amount over 25 cents. The machine keeps track of how much money has been inserted and there are 5 possible amounts it can hold (0, 5, 10, 15, 20 cents) before reaching 25 cents (the output state). Thus, the FSM as a Mealy machine, whose outputs depend on the current state and the input, will have 5 total states since there does not need to be a separate “dispense” state; the FSM will simply return to the start state after the total reaches or exceeds 25 cents.

VERILOG CODE

See attached files.



- The 7 anodes of the 7-segment LEDs connected to one common anode, with separate cathodes → segments illuminated by low logic, common anode activated by low logic too
 - **technically anode activated by high logic, BUT built-in inverter on each anode signal makes it also active low
- Dispense: For A2 as 'd' → AN2 low, f+ a high
- Return Nickel: For A4 as '5' → AN0 low, b+e high
- Return Dime: For A3:A4 as '10' → AN1 low, a+f+g+e+d high, AN0 low, g high
- Return 2 Dimes: For A3:A4 as '20' → AN1 low, f+c high, AN0 low, g high
- **segment LEDs [6:0] are [g:a], so binary b'7xxxxxxx is backwards

TROUBLESHOOTING

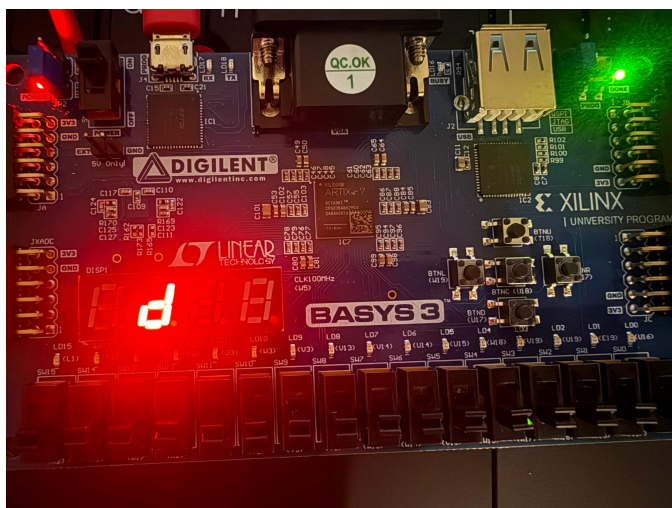
There was some trouble getting the LED segment digits to display different digits to represent the vending machine's outputs. The Basys3 manual stated that the common anode to enable each digit was active high, but upon further inspection it turned out to be active low since there is an inverter on each anode. In addition, Vivado TCL parsing is very nitpicky, and does not allow comments on the same line as the active constraints—this will cause errors during the bitstream generation process, as Vivado thinks the comments are part of the commands. Initially, I was trying to label the segment LEDs on the .xdc constraint file as 'a' through 'g' so that it matched the diagram in the Diligent manual. Also, when using the display segments to display the output, it is essential they stay on rather than blink momentarily so latch behavior must be implemented.

On the board during demo, there were some reliability issues since the outputs did not display as intended every time (it turns out that physical switches are messy). Physical switches bounce, and since the design was sampling raw switch changes without true debouncing, sometimes flicks were not captured as expected. This was because the input was not fully cleaned before being used, and thus a reset button was added. After btnC was added as a reset, if the vending machine got “stuck,” we could force the state back to S0.

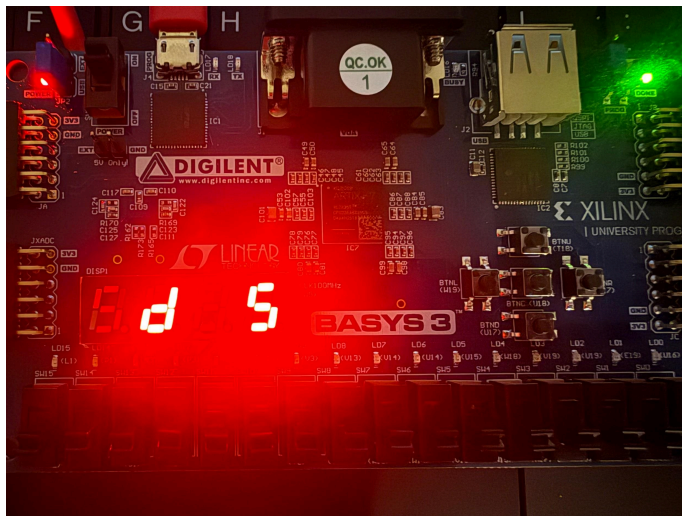
Throughout this lab, most of the issues were not with the vending machine logic itself, but with simulation setup, timing, and test expectations. Initially, syntax errors and incorrect file types prevented the testbench from running, causing Vivado to simulate only the design without inputs. After fixing that, the main issue was a timing mismatch: applying inputs on the clock edge caused unreliable coin detection, which was resolved by changing inputs on the negative edge. The largest source of errors came from incorrect test vectors that assumed the FSM reset between inputs, when in reality it maintains state continuously. Once the vectors were corrected to reflect proper state progression, all tests passed. The key takeaway is that correct timing and accurate test expectations are just as important as the FSM logic itself.

FPGA BOARD OUTPUT

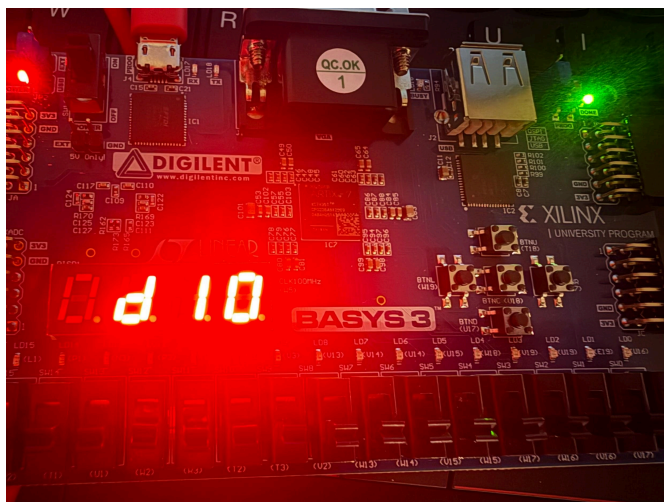
- Inputs: sw0=N, sw1=D, sw2=Q
- Outputs: AN2 as ‘d’=dispense, AN[0] as ‘5’=rN, AN[1:0] as ‘10’=rD, AN[1:0] as ‘20’=r2D
- Display LEDs update when switches are flipped down after being flipped up (mimicking inserting into coin slot)



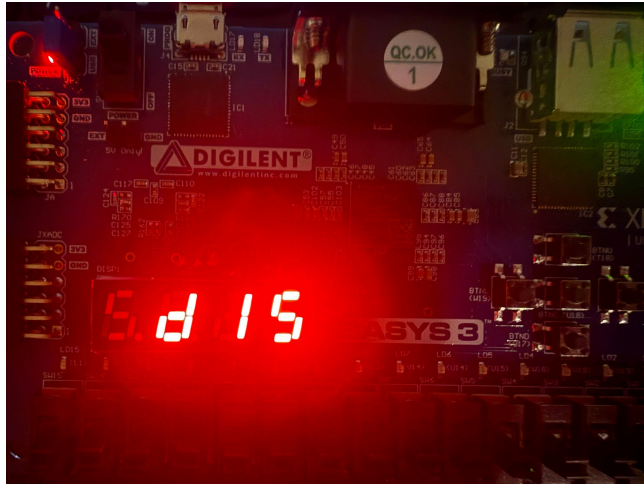
Input: Q (sw2 flipped up and then down)



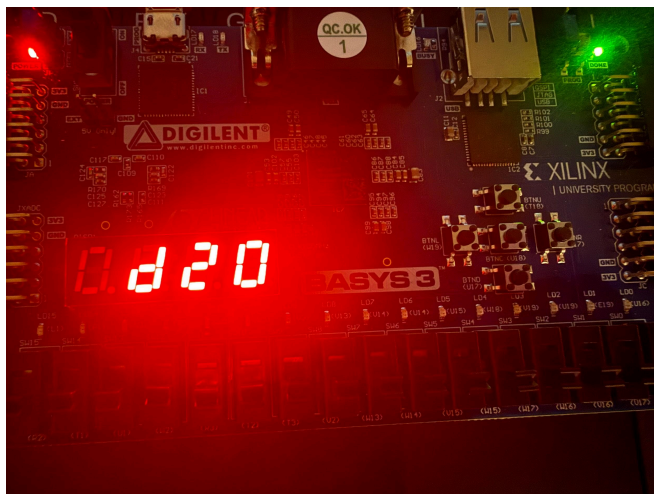
Input: N, then Q (sw0 flipped up-down, sw2 flipped up-down)



Input: D, then Q (sw1 flipped up-down, sw2 flipped up-down)



Input: $N+D$, then Q (sw0 flipped up-down, sw1 flipped up-down, then sw2 flipped up-down)



Input: $D+D$, then Q (sw1 flipped up-down twice, sw2 flipped up-down)